

EXAMEN D'ALGO-SDA - L3I**Exercice 1 : listes doublement chaînées (5.5 Pts)**

Un polynôme est constitué d'une somme de **monômes non nuls**. Chaque monôme est représenté par une structure composée de deux membres : degré (un entier) et coefficient (un réel).

Les polynômes seront représentés par une **liste doublement chaînée** de monômes non nuls (i.e : chaque cellule de la liste représente un monôme de coefficients non nuls) et **classés dans l'ordre croissant selon le degré**.

Pour faciliter la gestion des polynômes, on rajoutera deux adresses : **la première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste**.

La liste doublement chaînée représentant le polynôme est définie comme suit :

```
typedef struct {
    double coef ;
    int degre ;
} Monome ;

typedef struct {
    DListe tete;
    DListe queue;
} Polynome ;

typedef struct cellule {
    Monome monome ;
    struct cellule* suivant ;
    struct cellule* precedant ;
} *DListe ;
```

- 1) En parcourant le polynôme dans les deux sens ; **écrire en C** ; l'algorithme qui détermine l'adresse du monôme possédant le plus grand coefficient dans un polynôme non vide donné.
- 2) **Ecrire en C** ; l'algorithme d'**insertion en tête** d'un monôme donné dans un polynôme vide donné.
- 3) **Ecrire en C** ; l'algorithme d'**insertion en queue** d'un monôme donné dans un polynôme non vide donné.
- 4) Ecrire en langage algorithmique ou en C, un algorithme qui calcule le polynôme dérivé d'un polynôme P donné.

Note : le polynôme P n'est pas modifié après la dérivée.

$$P(x) = 3 + 5x^2 + 6x^3 - 2x^5 - 3x^8 \quad DP(x) = 10x + 18x^2 - 10x^4 - 24x^7$$

- 5) Ecrire en langage algorithmique ou en C, un algorithme qui calcule le polynôme représentant la somme de deux polynômes donnés selon le schéma suivant :

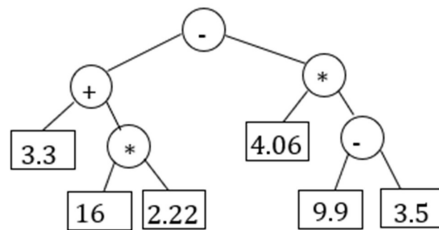
$$P1(x) = 3 + 5x^2 + 6x^3 - 2x^4 - 8x^5, \quad P2(x) = -3 + 7x - 4x^2 - 6x^3 + 7x^5 - 9x^8 + 3x^{10}$$

$$P1(x) + P2(x) = P(x) = 7x + x^2 - 2x^4 - x^5 - 9x^8 + 3x^{10}$$

Note : les polynômes P1 et P2 ne sont pas modifiés après la somme.

Exercice 2 Arbre binaire (4.5 Points)

On considère les expressions arithmétiques (**chaîne de caractères**) sur les réels n'utilisant que les opérateurs binaires +, -, *, /. Ces expressions peuvent être représentées par des arbres binaires dite aussi arbres d'expressions dont les nœuds internes sont étiquetés par l'un des quatre opérateurs tandis que les feuilles sont étiquetées par des nombres réels.



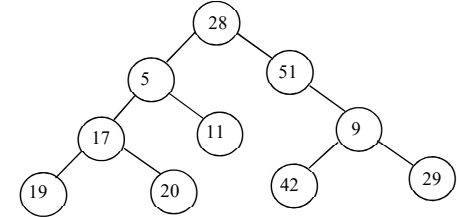
L'arbre d'expression représentant l'expression
Exp=((3.3+(16*2.22))-(4.06*(9.9-3.5)))

- 1) **Ecrire en C**, les structures de données nécessaires à la représentation d'un arbre d'expression.
- 2) **Ecrire en C**, un algorithme qui effectue l'évaluation d'un arbre d'expression donné.
- 3) Ecrire en langage algorithmique ou en C, un algorithme qui détermine le plus grand opérande présent dans un arbre d'expression non vide donné.
- 4) Ecrire en langage algorithmique ou en C, un algorithme qui détermine le nombre d'occurrence de chaque opérateur présent dans un arbre d'expression donné.

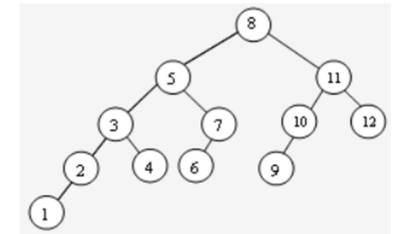
Exercice 3 : AVL (6.0 Pts)

Note : on suppose connu l'algorithme d'équilibrage.

- 1) **Ecrire en C**, l'algorithme d'insertion d'un élément donné dans un AVL donné.
- 2) Ecrire en langage algorithmique (ou en C), l'algorithme de construction d'un AVL à partir d'un arbre binaire donné parcouru selon le **parcours en largeur**.
- 3) Appliquer graphiquement (sous forme d'arbres) votre algorithme de construction à l'arbre binaire ci-contre.



- 4) Décrire le principe de suppression d'un élément donné dans un AVL donné.
- 5) **Ecrire en C**, l'algorithme de suppression d'un élément donné dans un AVL donné.
- 6) Appliquer votre algorithme à la suppression de l'élément 12 dans l'AVL ci-contre.

**Exercice 4 : Arbres Parfaits & File de Priorité (=Tas) (6.0 Pts)**

Une file de priorité (nommé Tas) permet de gérer un ensemble d'éléments selon une clé appelée leur priorité. Dans cet exercice, on confond chaque élément avec sa priorité.

On souhaite implémenter un arbre parfait ainsi qu'un Tas par un arbre binaire d'entiers dont la structure est définie comme ci-contre :

```
typedef int TElement;

typedef struct noeud {
    TElement donnee;
    struct noeud *filsG;
    struct noeud *filsD;
} *Arbre;
```

- 1) Soit le tas-max suivant représenté sous forme d'un tableau :

62	55	44	25	35	37	27	14	10	18
----	----	----	----	----	----	----	----	----	----

- a) Représenter graphiquement sous forme d'arbre binaire le tas-max ci-dessus.
 - b) Décrire graphiquement étape par étape, l'insertion de l'élément 60.
 - c) Décrire graphiquement étape par étape, la suppression du maximum dans le tas-max défini en (a).
- 2) En utilisant la **représentation chaînée d'un arbre binaire** définie ci-dessus : écrire en langage algorithmique (ou C), un algorithme qui vérifie si un arbre binaire donné est un arbre parfait ?
 - 3) En utilisant la **représentation chaînée d'un arbre binaire** définie ci-dessus : écrire en langage algorithmique (ou C), un algorithme qui vérifie si un arbre parfait donné est un tasMax ?

NOTES

- ♦ Les réponses aux questions doivent être claires et justifiées.
- ♦ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme les préconditions, ainsi que le passage des paramètres.
- ♦ **Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.**
- ♦ On suppose connues les primitives des piles et des files et donc vous pouvez l'utiliser sans les redéfinir.

*****Annexes*****

On suppose connu les algorithmes suivants :

Structure Polynôme :

Fonction degre : Monome → Entier
 Fonction coef : Monome → Reel
 Fonction donnee : DListe → Monome
 Fonction suivant : DListe → DListe
 Fonction precedent : DListe → DListe
 Fonction initDL : [] → DListe
 Fonction estVideDL : DListe → Booléenne
 Fonction tete : Polynome → DListe
 Fonction queue : Polynome → DListe
 Fonction initP : [] → Polynome
 Fonction estVideP : Polynome → Booléenne

Structure AVL :

Fonction donnee : AVL → TElement
 Fonction hauteur : AVL → Entier
 Fonction filsGauche : AVL → AVL
 Fonction filsDroit : AVL → AVL
 Fonction initA : [] → AVL
 Fonction estVideA : AVL → Booléenne
Fonction equilibrage : AVL → AVL

Structure File

Fonction sommetF : File → TElementF
 Fonction enfiler : TElementF x File → File
 Fonction defiler : File → File
 Fonction initF : [] → File
 Fonction estVideF : File → Booléenne
 Fonction estPleinF : File → Booléenne

Structure Pile

Fonction sommetP : Pile → TElementP
 Fonction empiler : TElementPx Pile → Pile
 Fonction depiler : Pile → Pile
 Fonction initP : [] → Pile
 Fonction estVideP : Pile → Booléenne
 Fonction estPleinP : Pile → Booléenne